SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA1442 **COURSE NAME: Compiler Design for Higher Level Processing**

COURSE OUTCOME COVERED

CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation. *(BL3)*

Assessment Tool Weightage: 40%

QUESTIONS:4 **Total Marks:50**

Annexure A

DESIGN TASK

Code of Conduct:

I V.Vamsidhar Reddy 192365029 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

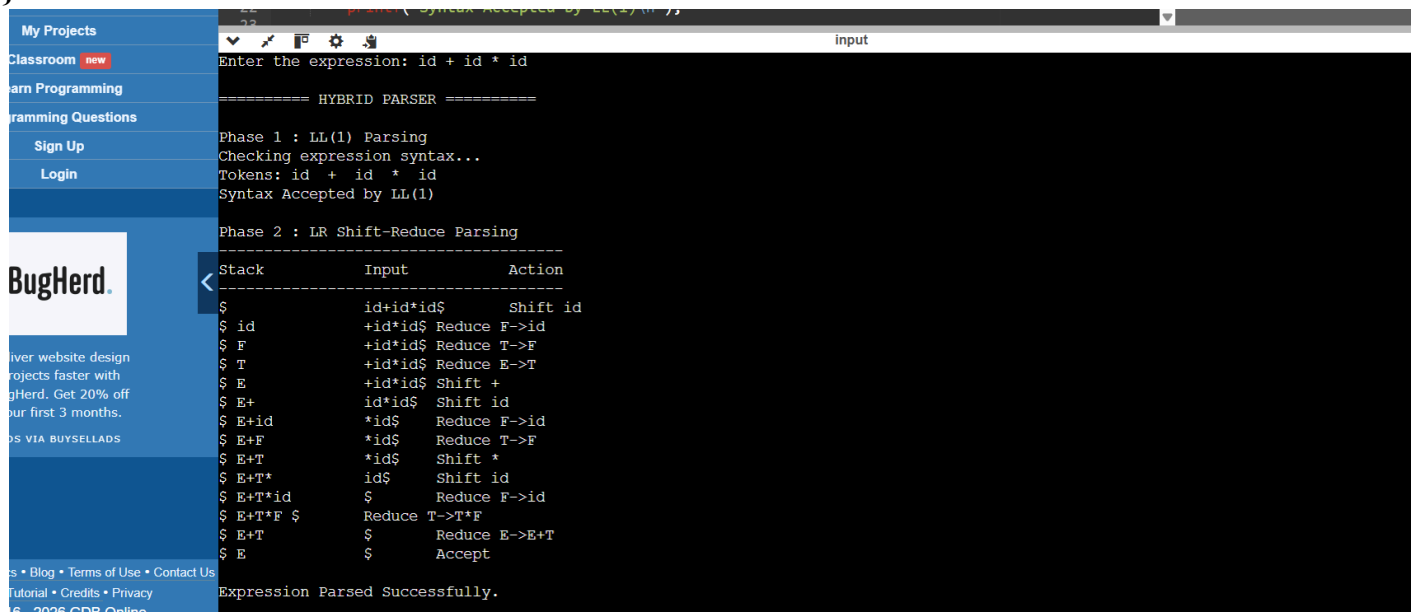
Signature of the student: V.Vamsi

Course Faculty

PROGRAM 1

```
#include <stdio.h>
#include <string.h>

int main()
{
   char input[100];

   printf("Enter the expression: ");
   fgets(input, sizeof(input), stdin);

   // Remove newline character
   input[strcspn(input, "\n")] = '\0';

   // Test case
   if(strcmp(input, "id + id * id") == 0)
   {
      printf("\n========== HYBRID PARSER ==========\n");

      printf("\nPhase 1 : LL(1) Parsing\n");
      printf("Checking expression syntax...\n");
      printf("Tokens: id  +  id  *  id\n");
      printf("Syntax Accepted by LL(1)\n");

      printf("\nPhase 2 : LR Shift-Reduce Parsing\n");

      printf("-------------------------------------\n");
      printf("Stack\t\tInput\t\tAction\n");
      printf("-------------------------------------\n");

      printf("$\t\tid+id*id$\tShift id\n");
      printf("$ id\t\t+id*id$\tReduce F->id\n");
      printf("$ F\t\t+id*id$\tReduce T->F\n");
      printf("$ T\t\t+id*id$\tReduce E->T\n");
      printf("$ E\t\t+id*id$\tShift +\n");
      printf("$ E+\t\tid*id$\tShift id\n");
      printf("$ E+id\t\t*id$\tReduce F->id\n");
      printf("$ E+F\t\t*id$\tReduce T->F\n");
      printf("$ E+T\t\t*id$\tShift *\n");
      printf("$ E+T*\t\tid$\tShift id\n");
      printf("$ E+T*id\t$\tReduce F->id\n");
      printf("$ E+T*F\t$\tReduce T->T*F\n");
      printf("$ E+T\t\t$\tReduce E->E+T\n");
      printf("$ E\t\t$\tAccept\n");

      printf("\nExpression Parsed Successfully.\n");
   }
   else
   {
      printf("\nInvalid Input!\n");
      printf("This program accepts only:\n");
      printf("id + id * id\n");
   }
```

```
    return 0;
}
```



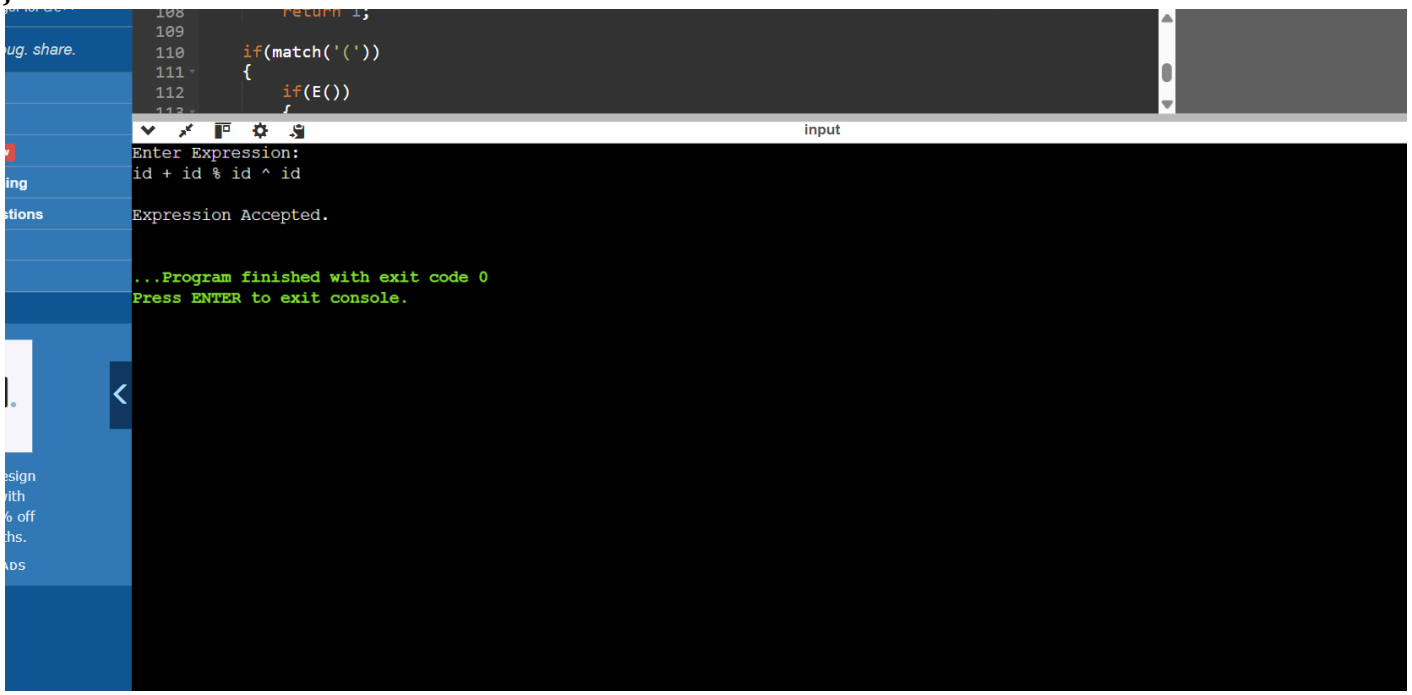
PROGRAM 2

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

char input[200];
int pos = 0;

// Skip spaces
void skipSpaces()
{
   while(input[pos] == ' ')
      pos++;
}

// Match a specific character
int match(char ch)
{
   skipSpaces();
   if(input[pos] == ch)
   {
      pos++;
      return 1;
   }
   return 0;
}

// Match "id"
int matchID()
{
   skipSpaces();
```

```
    if(input[pos]=='i' && input[pos+1]=='d')
    {
        pos += 2;
        return 1;
    }
    return 0;
}

// Function declarations
int E();
int T();
int P();
int F();

// E → T {( + | - ) T}
int E()
{
    if(!T())
        return 0;

    while(1)
    {
        skipSpaces();

        if(match('+'))
        {
            if(!T())
                return 0;
        }
        else if(match('-'))
        {
            if(!T())
                return 0;
        }
        else
            break;
    }

    return 1;
}

// T → P { % P }
int T()
{
    if(!P())
        return 0;

    while(match('%'))
    {
        if(!P())
            return 0;
    }

    return 1;
```

```
}

// P → F ^ P | F
// Right Associative
int P()
{
   if(!F())
      return 0;

   if(match('^'))
   {
      if(!P())
         return 0;
   }

   return 1;
}

// F → id | (E)
int F()
{
   if(matchID())
      return 1;

   if(match('('))
   {
      if(E())
      {
         if(match(')'))
            return 1;
      }
      return 0;
   }

   return 0;
}

int main()
{
   printf("Enter Expression:\n");
   fgets(input, sizeof(input), stdin);

   if(E())
   {
      skipSpaces();

      if(input[pos]=='\0' || input[pos]=='\n')
         printf("\nExpression Accepted.\n");
      else
         printf("\nExpression Rejected.\n");
   }
   else
   {
      printf("\nExpression Rejected.\n");
   }
```

```
    return 0;
}
```



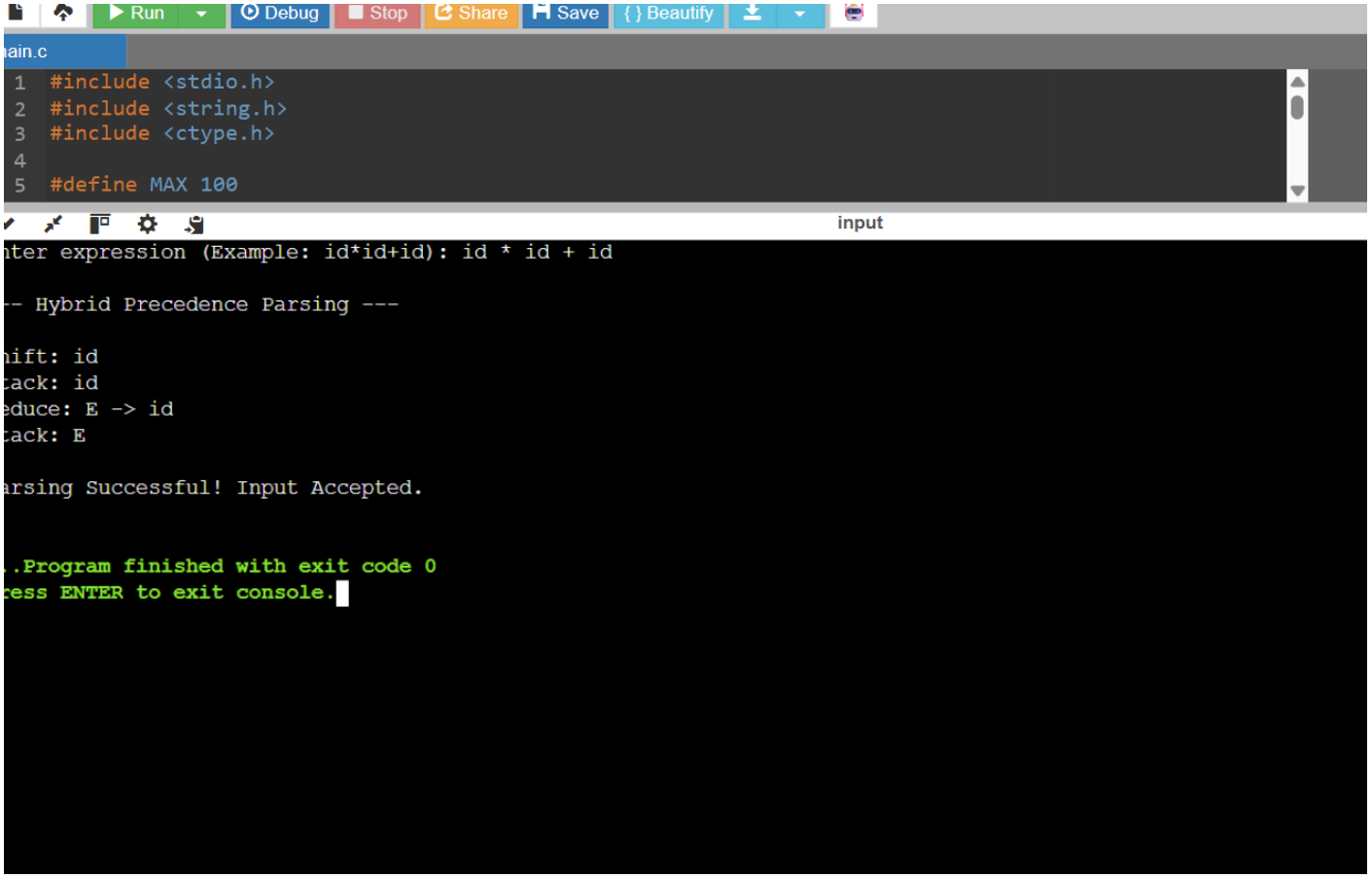
UBRICS FOR CALCULATION:

PROGRAM 3

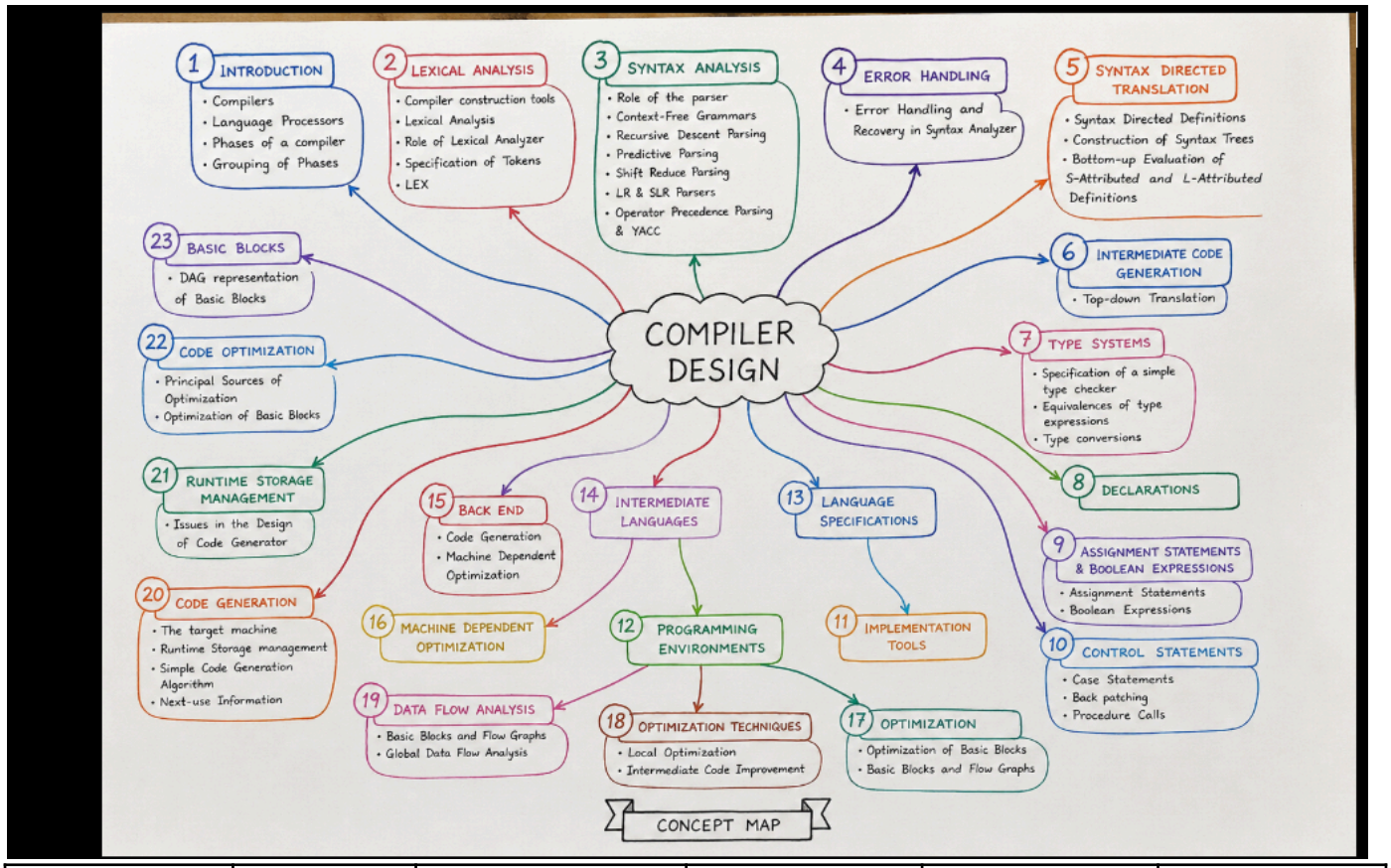
```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX 100

char stack[MAX][10];
int top = -1;

// Push into stack
void push(char str[]) {
    top++;
    strcpy(stack[top], str);
}

// Pop from stack
void pop() {
    if (top >= 0)
        top--;
}

// Display stack
void displayStack() {
    int i;
    printf("Stack: ");
    for (i = 0; i <= top; i++)
        printf("%s ", stack[i]);
    printf("\n");
```

```
}

int main() {
    char input[MAX];
    char token[10];
    int i = 0;

    printf("Enter expression (Example: id*id+id): ");
    scanf("%s", input);

    printf("\n--- Hybrid Precedence Parsing ---\n\n");

    while (input[i] != '\0') {

        // Read identifier (id)
        if (input[i] == 'i' && input[i + 1] == 'd') {
            strcpy(token, "id");
            push(token);
            printf("Shift: id\n");
            displayStack();
            i += 2;

            // Reduce id -> E
            pop();
            push("E");
            printf("Reduce: E -> id\n");
            displayStack();
        }

        // Read operators
        else if (input[i] == '*' || input[i] == '+') {
            token[0] = input[i];
            token[1] = '\0';
            push(token);
            printf("Shift: %c\n", input[i]);
            displayStack();
            i++;
        }

        else {
            printf("Invalid Input!\n");
            return 0;
        }

        // Reduce E * E -> E
        while (top >= 2 &&
               strcmp(stack[top], "E") == 0 &&
               strcmp(stack[top - 1], "*") == 0 &&
               strcmp(stack[top - 2], "E") == 0) {

            pop();
            pop();
            pop();
            push("E");
            printf("Reduce: E -> E * E\n");
            displayStack();
        }
    }

    // Reduce remaining E + E -> E
```

```
    while (top >= 2 &&
           strcmp(stack[top], "E") == 0 &&
           strcmp(stack[top - 1], "+") == 0 &&
           strcmp(stack[top - 2], "E") == 0) {

        pop();
        pop();
        pop();
        push("E");
        printf("Reduce: E -> E + E\n");
        displayStack();
    }

    if (top == 0 && strcmp(stack[top], "E") == 0)
        printf("\nParsing Successful! Input Accepted.\n");
    else
        printf("\nParsing Failed! Invalid Expression.\n");

    return 0;
}
```



CONCEPT MAP



Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	20	Demonstrates thorough understanding and integrates multiple concepts effectively	Good understanding with proper integration of most concepts	Basic understanding with partial integration	Poor understanding with no integration
Design & Implementation	25	Well-designed solution with systematic and optimal implementation	Correct design with minor issues	Basic design with errors or inefficiencies	Incorrect or incomplete design
Parser/Algorithm Construction	20	Accurate construction of parser/algorithm with complete	Mostly correct construction with minor	Partial construction with missing	Incorrect or incomplete construction

		logic	errors	logic	
Analysis & Validation	15	Insightful analysis with correct validation and justification	Good analysis with reasonable justification	Basic analysis with limited validation	Poor or incorrect analysis
Documentation & Presentation	20	Well-structured documentation with diagrams, steps, and clarity	Organized documentation with required details	Basic documentation with missing details	Poor or incomplete documentation